

Projekt jegyzőkönyv

Tárgy: Web technológiák 2. (GEIAL334-B2)

Projekt neve: Könyvtár Kezelő Rendszer

Név: Tóth Zsombor Gábor

Neptun-kód: D0H157

1. Bevezetés

A projekt célja egy teljes körű, böngészőből elérhető könyvtárkezelő webalkalmazás fejlesztése volt. A rendszer lehetővé teszi egy könyvtár készletének nyilvántartását, a könyvek kölcsönzését és visszavételét, valamint statisztikai adatok megjelenítését. Az alkalmazás elkülönített frontend és backend részből áll, amelyek REST API-n keresztül kommunikálnak egymással.

2. Célkitűzés

A fejlesztés során az alábbi főbb funkcionális követelményeket kellett, valamint szerettem volna teljesíteni:

- Felhasználói regisztráció és bejelentkezés JWT-alapú autentikációval
- Szerepköralapú hozzáférés-vezérlés (adminisztrátor és normál felhasználó)
- Könyvek listázása, keresése, szűrése és lapozása
- Könyvek hozzáadása, szerkesztése és törlése (csak adminisztrátor)
- Könyv kölcsönzése és visszaküldése
- Irányítópult statisztikákkal és diagramokkal

3. Alkalmazott technológiák

3.1 Frontend

A felhasználói felület **React 19** és **TypeScript** alapokon épül, a fejlesztői és build környezetet **Vite 8** biztosítja.

Az állapotkezelés két szinten valósul meg: a szerver oldali állapotokat (API adatok, cache) a **TanStack Query 5** kezeli, a helyi UI-állapotokat React beépített hook-jaival (**useState**, **useEffect**) kezelem. Az autentikációs állapotot egy **React Context** (**AuthContext**) tárolja, amely a felhasználói adatokat a böngésző **localStorage**-ában is megőrzi az oldalújrátöltések között.

Az űrlapok kezelésére a **React Hook Form** könyvtárat alkalmazom **Zod** sémaalapú validációval, amely mind a kliensoldali, mind a szerverre küldött adatok helyességét garantálja. A HTTP kéréseket **Axios** hajtja végre, egy közös interceptorral felszerelt instance-on keresztül, amely automatikusan csatolja az autentikációs tokent minden kéréshez.

A vizuális megjelenítéshez **Tailwind CSS 4** utility osztályokat, **Base UI** headless komponenseket és **Lucide React** ikonokat használok. Az animációkat **Framer Motion** biztosítja, az adatvizualizációt (kördiagram, oszlopdiagram) **Recharts** végzi. Az értesítési rendszer (toast üzenetek) **Sonner** könyvtárra épül.

3.2 Backend

A szerver oldal **Node.js** és **Express 5** keretrendszerre épül, szintén **TypeScript**-ben megvalósítva. Az adatokat **MongoDB** NoSQL adatbázisban tárolom, amelyhez a **Mongoose 9** ODM könyvtár biztosítja az adatmodellezést és a lekérdezési interfészt.

A felhasználói hitelesítés **JWT (JSON Web Token)** alapú: bejelentkezéskor a szerver aláírt tokent bocsát ki, amelyet a kliens minden védett kéréshez elküld. A jelszavak tárolása **bcryptjs** könyvtárral, szózással és hasheléssel történik. Az autentikációs végpontok brute-force védelme **express-rate-limit** middleware-rel valósul meg.

A bejövő kérések validációját szerver oldalon is **Zod** sémák végzik, amelyek egy általános **validate** middleware-en keresztül futnak le az útvonalkezelők előtt.

4. Rendszerarchitektúra

Az alkalmazás egy **kétrétegű (client-server) architektúrát** követ, amelyen belül a backend egy **háromrétegű (Controller–Service–Repository) mintát** valósít meg.

4.1 Backend rétegek

- **Routes:** Az Express útvonalak csupán a middleware-eket és a megfelelő controller metódust kötik össze.
- **Controllers:** HTTP kérés/válasz kezelők; kinyerik a szükséges adatokat a `req` objektumból és hívják a service réteget.
- **Services:** Tartalmazzák az üzleti logikát (pl. ellenőrzés, hogy van-e elérhető példány kölcsönzés előtt). Az egyes service osztályok a repository-kon keresztül érik el az adatbázist.
- **Repositories:** Kizárólag a Mongoose lekérdezéseket tartalmazzák, egyetlen felelősséggel: adatok olvasása és írása.

4.2 Frontend rétegek

- **Pages:** Legfelső szintű oldalkomponensek, amelyek adatot és eseménykezelőket adnak át az alsóbb komponenseknek.
- **Custom Hooks (useBooks):** A TanStack Query `useQuery` és `useMutation` hívásait csomagolják össze, egységes API-t nyújtva az oldalaknak.
- **Services (BookService, AuthService, BorrowService):** Axios-alapú HTTP hívások, egy közös `BaseService` ősből öröklve a hibakezelési logikát.
- **Components:** Kisebb, újrafelhasználható UI komponensek (pl. `BookTable`, `BookForm`, `BookFilters`, `Pagination`, `StatsGrid`, `GenreChart`, `YearChart`).

5. Adatmodellek

5.1 User (Felhasználó)

Mező	Típus	Leírás
username	String	Egyedi felhasználónév (min. 3 karakter)
password	String	Bcrypt-tel hashelt jelszó
role	'admin' 'user'	Szerepkör, alapértelmezés: 'user'
createdAt	Date	Mongoose timestamp

5.2 Book (Könyv)

Mező	Típus	Leírás
title	String	A könyv címe
author	String	Szerző neve
isbn	String	Egyedi ISBN-10 vagy ISBN-13 azonosító
publishedYear	Number	Megjelenés éve
genre	String	Műfaj
quantity	Number	Összes példányszám (≥ 0)
addedBy	ObjectId	Referencia a hozzáadó adminra
createdAt	Date	Mongoose timestamp

5.3 Borrow (Kölcsönzés)

Mező	Típus	Leírás
book	ObjectId	Referencia a kölcsönzött könyvre
user	ObjectId	Referencia a kölcsönző felhasználóra
borrowDate	Date	Kölcsönzés dátuma
dueDate	Date	Visszahozás határideje (alapért. 14 nap)
returnDate	Date	Tényleges visszahozás dátuma
status	'active' 'returned'	Kölcsönzés státusza

6. API végpontok

6.1 Autentikáció (/api/auth)

Metódus	Végpont	Leírás	Védelem
POST	/register	Új felhasználó regisztrációja	Rate limit
POST	/login	Bejelentkezés, JWT token visszaadása	Rate limit

6.2 Könyvek (/api/books)

Metódus	Végpont	Leírás	Védelem
GET	/	Könyvek listázása (szűrőkkel, lapozással)	JWT
POST	/	Új könyv hozzáadása	JWT + Admin
PUT	/:id	Könyv adatainak módosítása	JWT + Admin
DELETE	/:id	Könyv törlése	JWT + Admin

A GET / végpont az alábbi query paramétereket fogadja: `page`, `limit`, `sortBy`, `order`, `search`, `genre`, `all`.

6.3 Kölcsönzések (/api/borrows)

Metódus	Végpont	Leírás	Védelem
POST	/:id/borrow	Könyv kölcsönzése	JWT
POST	/:id/return	Könyv visszahozása	JWT
GET	/my	Saját kölcsönzések listája	JWT
GET	/active	Összes aktív kölcsönzés	JWT + Admin

7. Főbb funkciók megvalósítása

7.1 Autentikáció és szerepköralapú hozzáférés

A bejelentkezés után a szerver egy 30 napig érvényes JWT tokenet állít ki. A token a böngésző `localStorage`-ában tárolódik. Az Axios kérés interceptor minden API híváshoz automatikusan hozzáfűzi a `Authorization: Bearer <token>` fejléct. A válasz interceptor 401-es hibakód esetén automatikusan kijelentkezteti a felhasználót.

A backend két middleware-t alkalmaz a védett végpontokon: `protect` (érvényes JWT ellenőrzése) és `admin` (adminisztrátor szerepkör ellenőrzése).

7.2 Könyvlista és szűrés

A könyvlista oldal (`Books.tsx`) szerver oldali szűrést, rendezést és lapozást alkalmaz. A keresőmező `debounce`-olva van (500 ms), hogy a szerver ne kapjon feleslegesen sok kérést gépelés közben. A rendezési és szűrési állapotok a komponens `filters` state objektumában tárolódnak, és változásuk automatikusan új API lekérést vált ki a TanStack Query cache mechanizmusán keresztül.

7.3 Kölcsönzési logika

A szerver kölcsönzéskor ellenőrzi:

- létezik-e a könyv,
- van-e elérhető szabad példány (`quantity - activeborrows.length > 0`),
- nincs-e a felhasználónak már aktív kölcsönzése ugyanabból a könyvből.

A könyv lejáratát a `dueDate` mező tárolja (kölcsönzéstől számított 14 nap). Az irányítópult a lejárt, de még vissza nem hozott kölcsönzéseket `overdue`-ként jelzi.

A `GET /api/books` válasz minden könyvhöz mellékeli az aktív kölcsönzési rekordokat és a kiszámított `availableQuantity` értéket, így a frontend egyetlen kérésből megkapja a teljes állapotot.

7.4 Irányítópult és vizualizáció

A `Dashboard` oldal az összes könyvet betölti (`all=true` paraméterrel), majd kliensoldali aggregációval számítja ki a statisztikákat: összes példány, aktívan kölcsönzött, saját kölcsönzések, lejárt tételek. Az adatokat egy kördiagram (műfaj szerinti megoszlás) és egy oszlopdiagram (kiadási évek szerinti eloszlás) is szemlélteti.

8. Adatbázis inicializálás (Seed)

A fejlesztési és tesztelési célokat egy seed szkript (`seed.ts`) szolgálja, amely:

- törli a meglévő adatokat,
- létrehoz 1 adminisztrátor és 4 normál felhasználói fiókot,
- feltölt 50 könyvet különböző műfajokból,
- generál aktív és visszahozott kölcsönzési rekordokat, köztük lejárt tételekkel.

A szkript futtatása: `npm run seed` a `server` könyvtárból.

9. Környezeti konfiguráció

Backend (`.env`)

```
PORT=5000
MONGODB_URI=mongodb://localhost:27017/library_system
JWT_SECRET=<titkos_kulcs>
NODE_ENV=development
```

Frontend (`.env`)

```
VITE_API_URL=http://localhost:5000/api
```

10. Az alkalmazás futtatása

```
# Backend indítása
```

```
cd server
npm install
npm run dev
```

```
# Frontend indítása (új terminálban)
```

```
cd client
npm install
npm run dev
```

A frontend alapértelmezetten a `http://localhost:5173`, a backend a `http://localhost:5000` porton érhető el.

11. Összefoglalás

A projekt megvalósítása során egy modern, teljes körű webalkalmazás készült el, amely jól elkülönített frontend és backend rétegekből áll. A backend háromrétegű (Controller–Service–Repository) architektúrája biztosítja a kód karbantarthatóságát és tesztelhetőségét. A frontend a TanStack Query cache mechanizmusával minimalizálja a felesleges API hívásokat, a Zod sémák kliensen és szerveren egyaránt validálják az adatokat, a szerepköralapú hozzáférés-vezérlés pedig mind UI, mind API szinten érvényesül.